

MBAX 6418

Serving and Debugging

David Dobolyi, PhD
Assistant Professor
Stone Family Faculty Scholar
david.dobolyi@colorado.edu



Leeds School of Business
UNIVERSITY OF COLORADO **BOULDER**



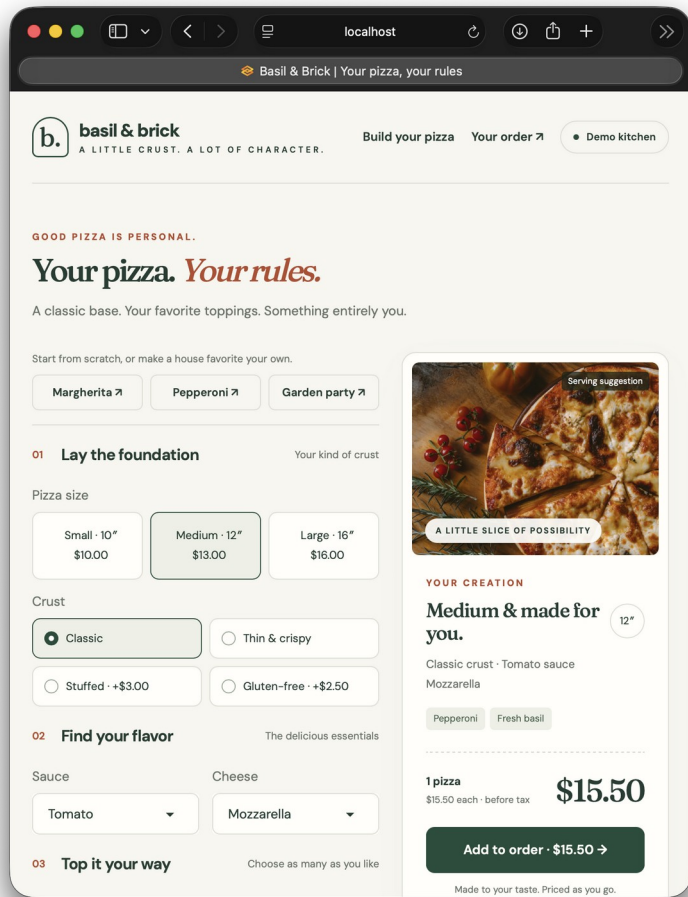
Topics

- Building, Serving, and Sharing
- Incorporating APIs
- Debugging
- GitHub

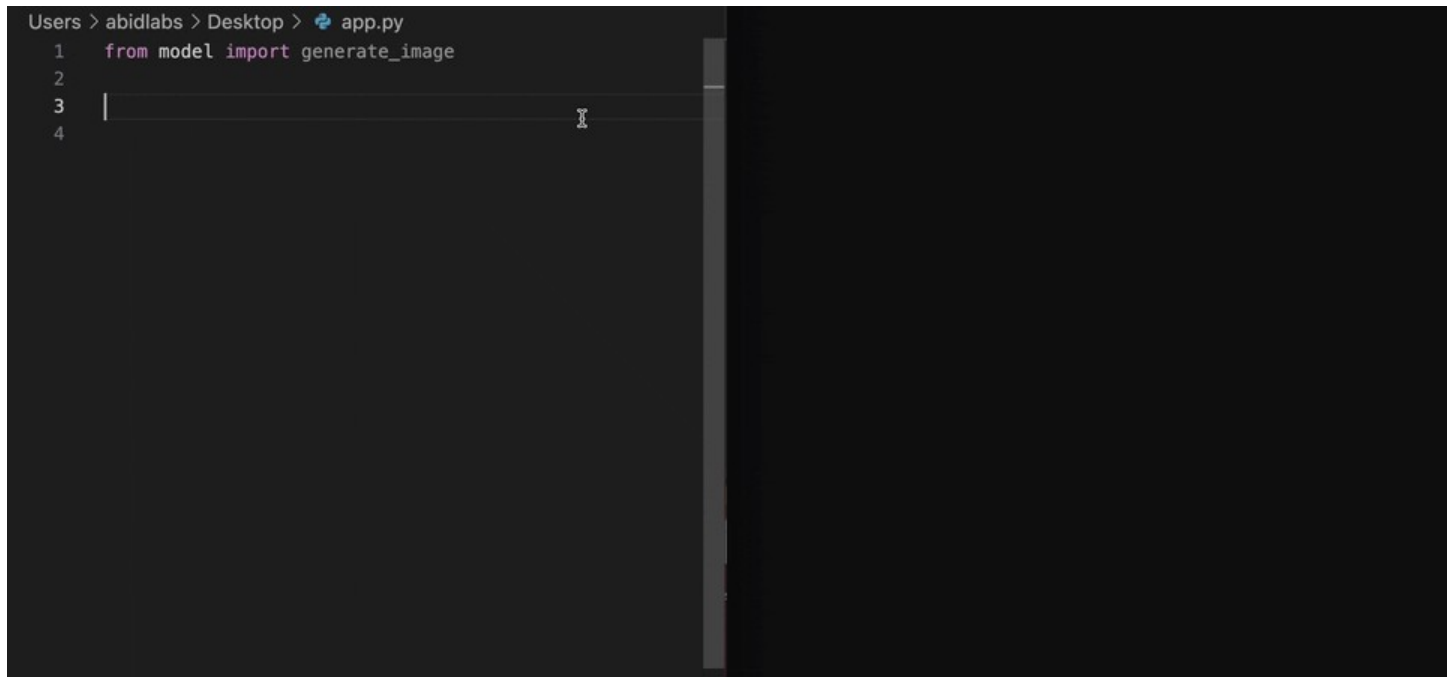
Building, Serving, and Sharing

Building

- Let's create a small web app, add features (including one with AI), and share it
- We'll also commit it to GitHub for version control
- **Starter prompt:**
 - “build a python app using gradio for a pizza ordering website where customers can customize their pizzas with things like toppings, size, crust, etc.”



Choosing an Interface

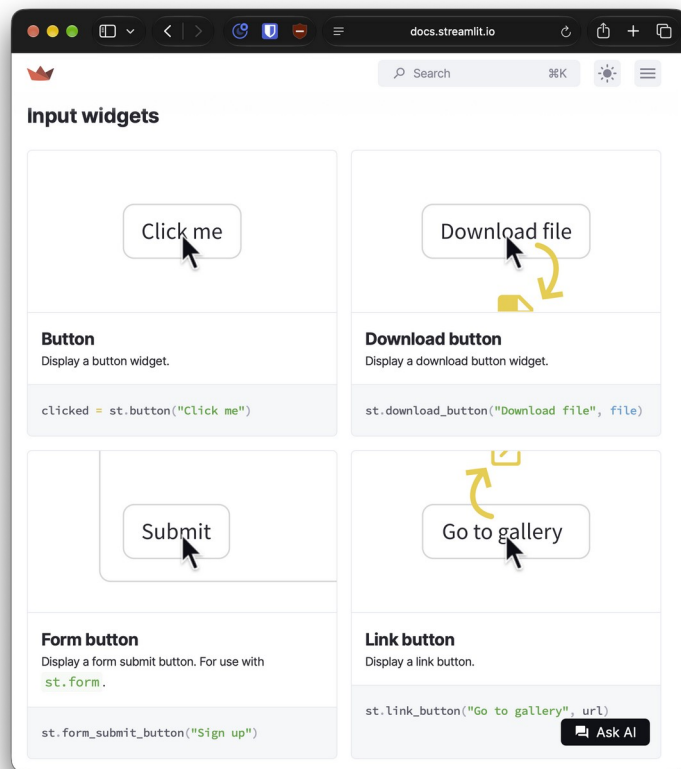


```
Users > abidlabs > Desktop > app.py
1  from model import generate_image
2
3  |
4
```

- We have many interface options, but for Python, *Gradio* and *Streamlit* are good, **high abstraction** options (i.e., they handle many of the technical details for us)

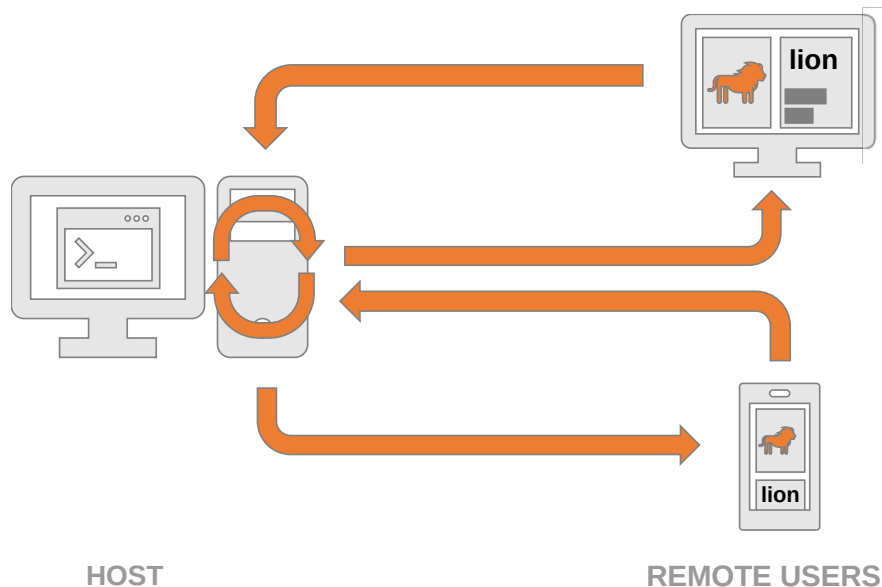
Components and Widgets

- Gradio and Streamlit provide prebuilt user interface (UI) elements (e.g., buttons, sliders, and text inputs) to help you quickly build interactive Python apps:
- [Gradio Components](#)
- [Streamlit Elements/Widgets](#)



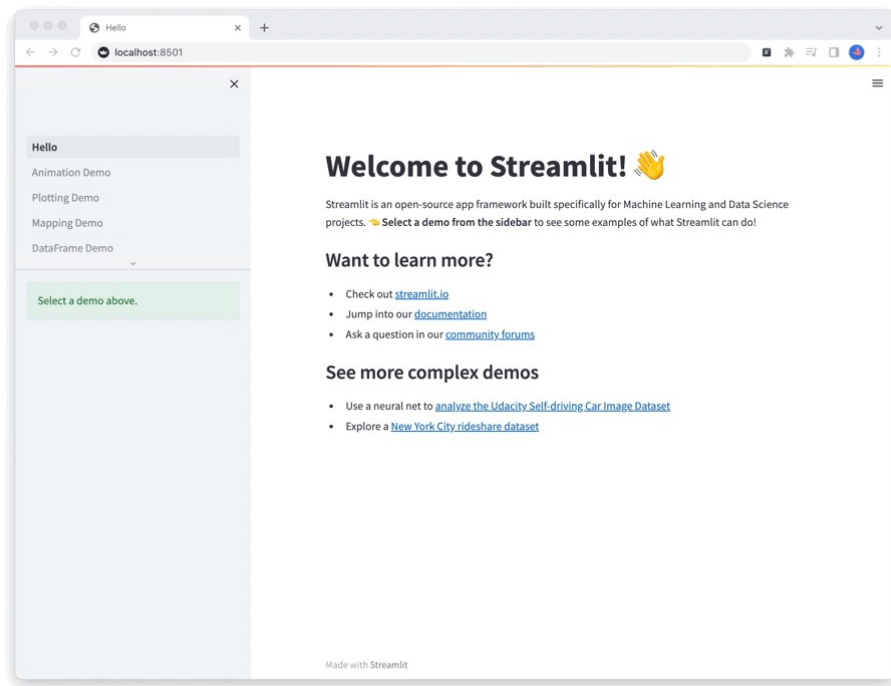
Serving

- Gradio provides easy serving without web hosting
- You can open your app locally using the default port, 7860:
 - <http://127.0.0.1:7860>
 - <http://localhost:7860>
- Adding `share=True` to `launch()` provides a temporary link, e.g., 60de1eaeae5f2b135.gradio.live/
- You must keep your computer running so others are able to load your app in their browser



Gradio vs. Streamlit vs. ...

- Gradio is one Python-based option, and Streamlit is another
- Both have their pros and cons, and each is better at some tasks vs. others
 - Chatbots, RAG apps, model interfaces, etc.
- Ultimately, use whatever you think works (or looks) best for your task (and explore alternatives)



Incorporating APIs

Application Programming Interface (API)

- **Application programming interfaces (APIs)** provide a structured way for software applications to talk to each other
- Your app sends a request and receives a response
- We'll use APIs to allow our apps to do things like:
 - Talk to large language models (LLMs) to generate text, power chatbots, classify data, etc.
 - Interpret or generate images
 - Transcribe or generate audio/video
 - Search the web via search APIs or tools (including MCP)

OpenAI API

- For example, OpenAI provides developers an [API Platform](#) with all kinds of models
- You pay per request (e.g., input/output price per million tokens, or MTok)
- The [OpenAI Python API library](#) is one way to connect

Flagship models

Compare model capabilities and specifications.

[View all](#)

[Compare models](#)



GPT-6 Astra

Our most capable model, built for the hardest end-to-end work

Model ID `gpt-6-astra`

Reasoning

Input price **\$10 / Input MTok**

Output price **\$50 / Output MTok**



GPT-5.6 Sol

Flagship model for complex professional work

Model ID `gpt-5.6-sol`

Alias `gpt-5.6`

Reasoning

Input price **\$4 / Input MTok**

Output price **\$20 / Output MTok**



GPT-5.6 Terra

GPT-5.6 model that balances intelligence and cost

Model ID `gpt-5.6-terra`

Reasoning

Input price **\$2 / Input MTok**

Output price **\$12 / Output MTok**



GPT-5.6 Luna

GPT-5.6 model optimized for cost-sensitive workloads

Model ID `gpt-5.6-luna`

Reasoning

Input price **\$0.20 / Input MTok**

Output price **\$120 / Output MTok**

Image

Models for image generation and editing



GPT-Image-2.5 Sunburst

Our most capable model for image generation and editing



GPT-Image-2.5 Flare

Fast, high-quality everyday image generation

Realtime

Models for realtime speech and translation



GPT-Realtime-2.1

Reasoning model with tool use



GPT-Realtime-2

Reasoning model with tool use



GPT-Realtime-1.5

The best voice model for audio in, audio out



GPT-Realtime-2.1 Mini

Reasoning model with tool use



GPT-Realtime-Translate

Streaming speech-to-speech translation model



GPT-Realtime Mini

A cost-efficient version of GPT-Realtime

Deprecated

OpenAI-Compatible Endpoints

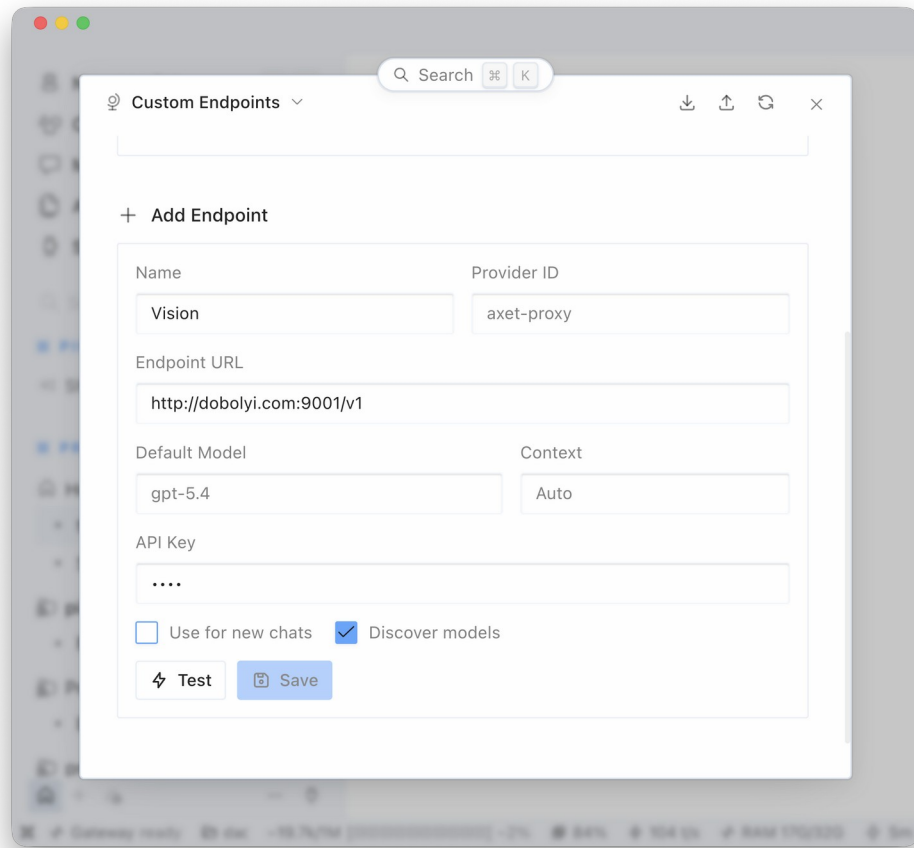
- The OpenAI Python API library is designed to work with OpenAI models, but you can also use it to talk to **OpenAI-compatible endpoints**
- These endpoints communicate the same way as the official OpenAI API Platform, but can be hosted elsewhere
- For example, we are already using a free, OpenAI-compatible endpoint for our class LLM (served via vLLM):
 - **Base URL:** <http://dobolyi.com:9000/v1>
 - **API Key:** 6418
 - **Model Name:** DeepSeek-V4-Flash-0731

More Endpoints

- Throughout this class, I will provide more endpoints to help with various tasks
- For example, here is a new endpoint we can use for augmenting vision and for classification tasks:
 - **Base URL:** <http://dobolyi.com:9001/v1>
 - **API Key:** 6418
 - **Model Name:** cyankiwi/Qwen3.6-35B-A3B-AWQ-4bit
- We'll also talk about models for embeddings, multimodal generation, etc. in future classes

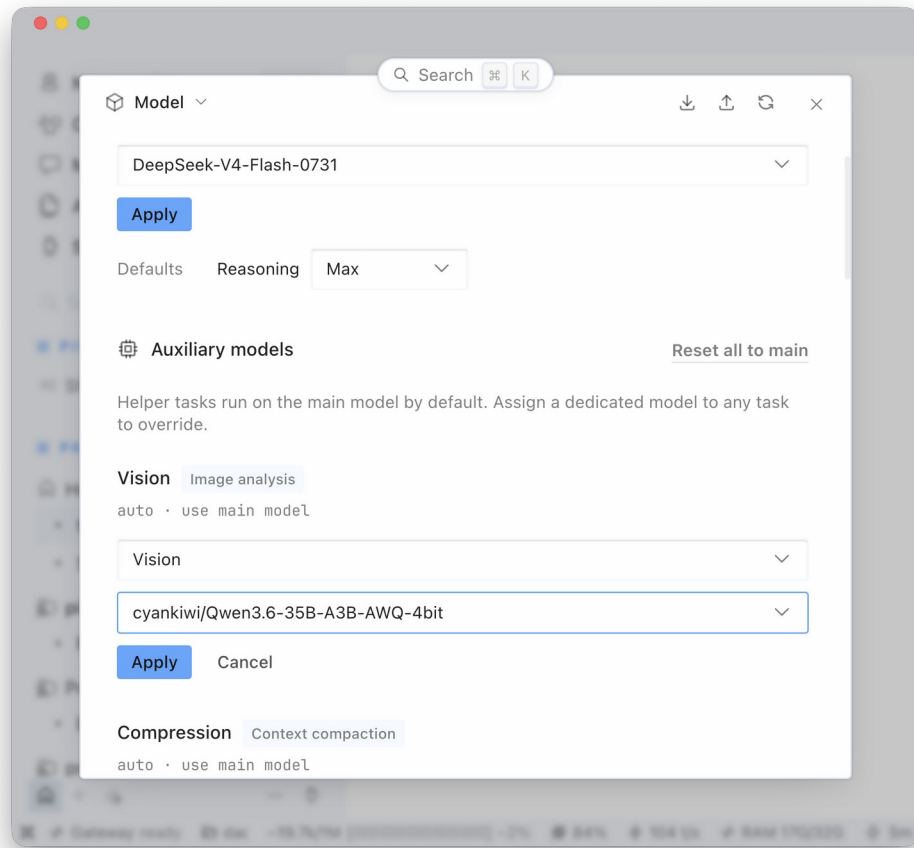
Adding a Vision Model to Hermes Agent

- **Hermes Desktop:** Go to Settings → Providers → Custom Endpoints, fill out the screen as shown on the right (be sure to uncheck “Use for new chats”), click ⚡ Test followed by Save
- **Hermes CLI:** In the terminal, run *hermes model*, find “Custom endpoint (enter URL manually)” and follow the instructions
- Or ask Hermes to set up a new vision model provider for you



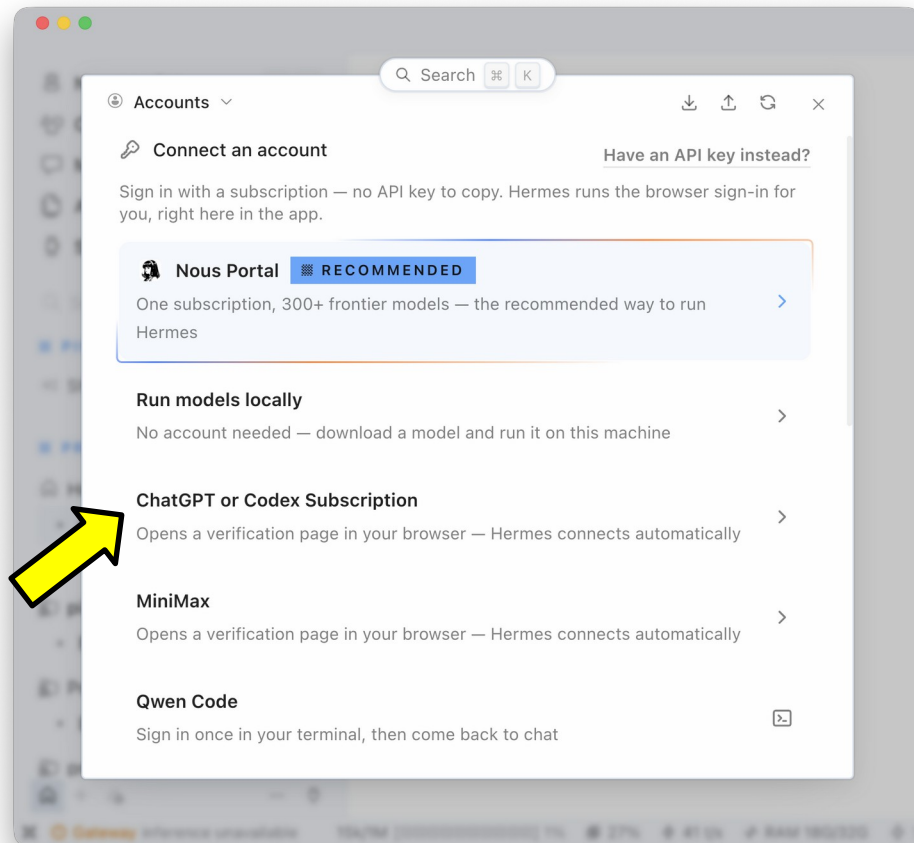
Changing the Auxiliary Vision Model

- **Hermes Desktop:** Go to Settings → Model, find “Vision” under “Auxiliary models”, click Change and fill out the screen as shown on the right (be sure to click Apply)
- **Hermes CLI:** In the terminal, run *hermes model*, find “Configure auxiliary models...” and follow the instructions



Adding Providers / Subscriptions

- You can also add subscriptions (e.g., Codex) to Hermes Agent
 - **Hermes Desktop:** Go to Settings → Providers → Accounts, click “Other providers”, choose an option, and follow the instructions
 - **Hermes CLI:** In the terminal, run *hermes model*, select an option (e.g., “OpenAI ► ...”) and follow the instructions
 - Or ask Hermes for help



Debugging

Bugs and Bug Reports

- While you build and test your app, your code will invariably contain **bugs**, which are errors in the software that cause it to fail or not work as intended
- To fix bugs, you need to create a thorough **bug report**, which helps document, reproduce, and ultimately fix each issue
- **Warning!** Just because an agent says a bug was fixed doesn't mean it's true! You must check!



Writing Useful Bug Reports

- **Key Questions**

- What is the problem?
- Where in the app does it happen?
- What did you expect to happen?
- What actually happened?
- *How do you reproduce it?*

- **Supporting Evidence**

- Was there an **error message**?
- Can you provide a screenshot?
- Does it happen every time or only sometimes?



Working with Hermes to Fix Bugs

- Provide each bug report to Hermes and ask for a **targeted, testable fix**
- Verify every fix by trying your reproduction steps: no matter what Hermes claims, *test, retest, and test again*
- Check related features: fixing one bug can introduce others
- To make repeated testing easier, consider asking Hermes for **automated tests** (as opposed to your own **manual tests**), including:
 - **Unit tests** – tests for small, specific parts of your software
 - **Smoke tests** – quick/inexpensive tests that ensure your app starts/runs and the basic functions work
 - **End-to-end tests** – perform and verify a complete workflow

GitHub

What is GitHub?

- GitHub's first [Start Your Journey](#) tutorial asks, [“What is GitHub?”](#):
 - “GitHub is a platform that supports your entire software development process, from planning work to deploying and operating software.”
- GitHub is built on the open-source **version control** system called [Git](#), which records changes to project files
- We'll use Git and GitHub primarily to:
 - Manage our code
 - Recover earlier versions
 - Collaborate with others
- All of this will happen inside a **repository (or repo)**, which stores a project's files, its history, privacy/collaboration settings, etc.

Working with GitHub

- Before creating a repo on GitHub, you need an account:
 - <https://github.com/signup>
- With an account, you can create your first repo via the [web site](#) or locally using the [GitHub Desktop](#) application (which you can later *push* to GitHub)
- Or just ask Hermes:
 - “Help me connect to my GitHub account”
 - “Create and connect a new GitHub repo for this project before we begin”

Hermes and GitHub

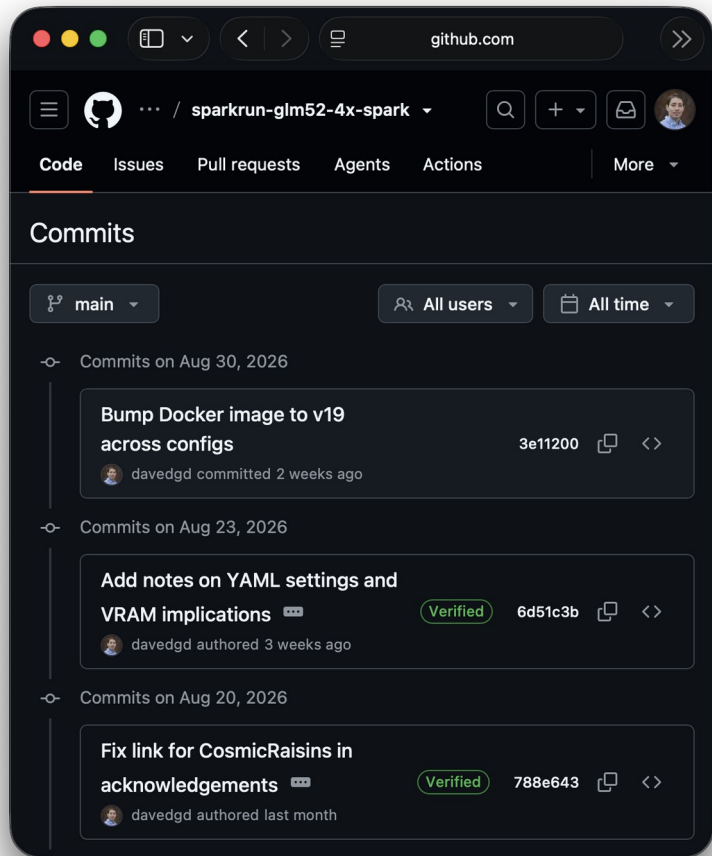
- If you ask it to (e.g., “Let’s use GitHub for version control”), Hermes will take care of Git/GitHub on your behalf, including common commands:
 - **git init** – create a new local repository
 - **git clone** – copy an existing repository to your computer
 - **git status** – show local changes and what’s ready to commit
 - **git add** – select changes to include in the next commit
 - **git commit** – record selected changes as a checkpoint
 - **git push** – send commits to a remote repository, e.g., GitHub
 - **git revert** – create a new commit that reverses a previous commit
 - **git pull** – retrieve/incorporate remote changes to your local copy
 - **git diff** – compare changes between file versions

Commits

- Ask Hermes to create a **commit** with each meaningful change and consider grouping related changes into checkpoints, especially when debugging or experimenting with **user interface (UI)** styling:
 - “Review these changes and commit them with a clear summary”
- Hermes will write commit messages for you which include:
 - **Summary (Subject)** – a summary of changes (50 characters or fewer [recommended](#))
 - **Description (Body)** – optional details of what changed and why
- Ask Hermes to **push** regularly, or specify when to wait:
 - “Push all of the latest commits” or...
 - “Hold off on pushing until I say so”

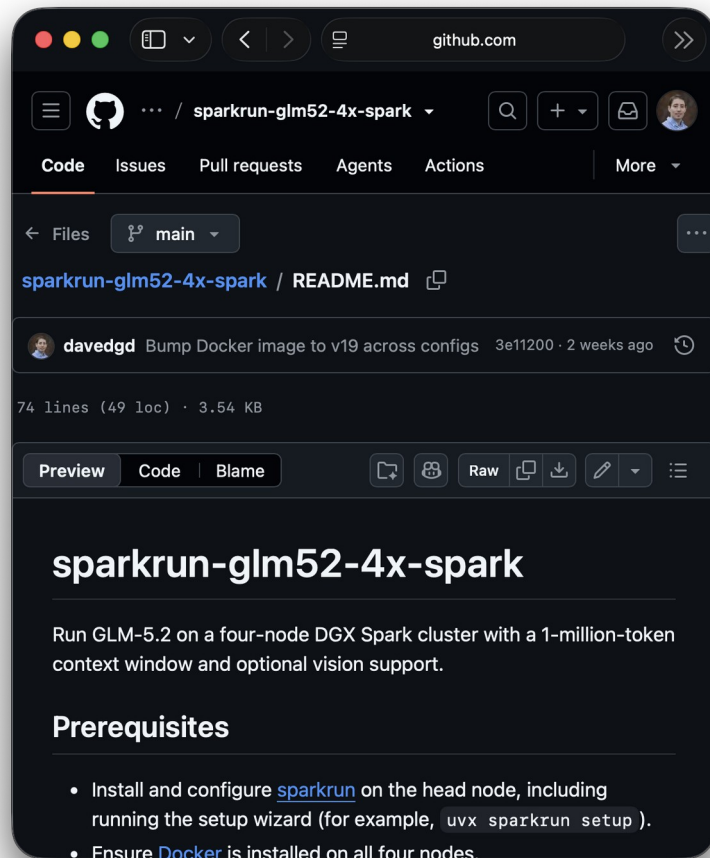
Reverting Commits

- Commits are useful for tracking and recovering from mistakes
- If a new bug appears, work with Hermes to review your commit history on GitHub to help identify which change introduced it
- Ask Hermes to **revert** the relevant commit, undoing the changes:
 - “This worked before the last commit: find and undo what broke it”
- Test again to confirm the problem is actually resolved



README.md

- When you create a repo or make your first commit, include a **README.md**, which describes the project, how to set it up, and how to use it
- Update this file as your project changes
- Hermes will typically create this file for you, but you can also ask it to directly:
 - “Create a README.md explaining this project and how to use it”



Branching and Merging

- Use **branch** to create separate lines of work in a repo to safely:
 - Develop a new feature
 - Fix a bug
 - Experiment
- The default branch is typically called **main**, and it holds the primary version you build from
- You can create and name other branches (e.g., dev)
- Use **merge** to combine changes across branches (e.g., dev into main)
- Ask Hermes:
 - “Create a branch for this feature and test it before merging into main”

Collaboration: Issues and PRs

- Create **issues** to provide a record of tasks, discussions, and progress, giving you, collaborators, and Hermes useful context
- Create **pull requests (PRs)** to propose changes you want to merge (e.g., fixes to issues), allowing for review, discussion, and tracking
 - Collaborators can suggest revisions before you merge
- Ask Hermes:
 - “Create an issue from this bug report”
 - “Open a pull request for this feature and summarize the changes and testing”